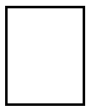

NIMBU OS

A Minimal Linux Kernel OS

PROJECT DOCUMENTATION

SETU Project No. 10 | Software Domain

Zoho Nagpur | January 2026

Version 1.0

For Zoho SETU Evaluation Only

1. Project Overview

Project Title	NIMBU OS — Baseline Kernel OS
Project No.	10
Domain	Software — Linux
Initiative	ZOHO SETU (Student's Engagement for Transformative Upskilling)
Organized By	Zoho Nagpur
Version	1.0
Date	January 2026

1.1 Abstract

NIMBU OS is a minimal, from-scratch Linux-based operating system developed as part of the ZOHO SETU Project-Based Learning initiative. The project aims to build a minimal Linux kernel with core components including device drivers, memory management, process scheduling, and a functional basic shell environment. The name NIMBU (Hindi for lemon) reflects the project's philosophy: clean, zesty, and refreshingly minimal — much like a lemon's sharp clarity amidst complexity.

1.2 Project Objectives

- Build a minimal Linux kernel from source with essential subsystems
- Implement core kernel components: memory manager, process scheduler, interrupt handlers
- Develop basic device drivers for keyboard, VGA display, and serial port
- Create a functional shell with basic command execution support
- Implement a simple bootloader with a custom NIMBU OS boot menu featuring the lemon branding
- Document architecture, design decisions, and implementation details thoroughly

1.3 Scope

The scope of NIMBU OS covers the complete lifecycle of a minimal operating system — from bootloader initialization to a running shell prompt. The system targets x86 architecture and is developed on Linux. The project emphasizes correctness and understanding of Linux internals over feature breadth.

2. System Architecture

2.1 High-Level Architecture

NIMBU OS follows a monolithic kernel architecture, where all core OS services run in kernel space. The system is structured in clearly defined layers:

Layer	Component	Description
Layer 0 — Bootloader	GRUB / Custom MBR	Loads kernel image, sets up boot menu with NIMBU OS branding
Layer 1 — Kernel Core	Entry Point (boot.asm)	Switches CPU to protected mode, sets up GDT/IDT
Layer 2 — Memory	PMM + VMM + Heap	Physical/virtual memory management with paging enabled
Layer 3 — Interrupts	IDT + ISR + IRQ	Interrupt descriptor table, handlers, and hardware IRQ routing
Layer 4 — Drivers	Keyboard, VGA, Serial	Hardware abstraction for I/O devices
Layer 5 — Shell	nimbu-shell	Command-line interface for user interaction

2.2 Kernel Components

2.2.1 Bootloader & Boot Menu

NIMBU OS uses a two-stage boot process. The first stage is a minimal Master Boot Record (MBR) loader written in x86 assembly. The second stage presents the NIMBU OS boot menu — a custom text-mode interface displaying the lemon logo (□) and system options.

Figure 1: NIMBU OS Boot Menu Lemon Logo

The boot menu displays the following screen upon system startup:

```
  NIMBU OS v1.0 — Minimal Linux Kernel
========================================

[1] Boot NIMBU OS (default)
[2] Boot in Debug Mode
[3] Memory Diagnostic
[4] Reboot
```

```
Press [1-4] or wait 5 seconds for default boot...
```

2.2.2 Memory Management

The memory subsystem implements both physical and virtual memory management:

- Physical Memory Manager (PMM): Bitmap-based page frame allocator tracking 4KB pages across all RAM
- Virtual Memory Manager (VMM): Page-directory and page-table management with 32-bit paging
- Kernel Heap: slab-like allocator supporting kmalloc() and kfree() for dynamic kernel memory
- Higher-Half Kernel: Kernel mapped at 0xC0000000 (3 GB virtual) following the higher-half convention

2.2.3 Process & Scheduler

NIMBU OS implements a basic preemptive round-robin scheduler with the following characteristics:

- Process Control Block (PCB): Stores registers, stack pointer, page directory, and state
- Scheduler: Timer-driven (PIT at 100 Hz) preemptive context switching
- States: RUNNING, READY, BLOCKED, ZOMBIE
- System calls: int 0x80 software interrupt interface

2.2.4 Interrupt Handling

The IDT (Interrupt Descriptor Table) is configured for all 256 interrupt vectors:

- ISRs (Interrupt Service Routines) 0–31: CPU exception handlers (divide-by-zero, page fault, GPF, etc.)
- IRQs 32–47: Hardware interrupts remapped via 8259 PIC (timer, keyboard, serial, etc.)
- Syscall vector 128 (0x80): User-space to kernel-space gate

2.2.5 Device Drivers

Driver	Interface	Functionality
VGA Text Driver	Memory-mapped I/O (0xB8000)	80x25 text mode, color support, cursor control
PS/2 Keyboard Driver	IRQ1 / I/O Port 0x60	Scancode-to-ASCII translation, special key handling
Serial (UART) Driver	IRQ4 / I/O Port 0x3F8	Debug output, COM1 at 115200 baud
PIT Timer Driver	IRQ0 / I/O Port 0x40	System tick at 100Hz for preemptive scheduling

2.2.6 NIMBU Shell

The nimbu-shell is a minimal command-line interface that loads after kernel initialization. It provides:

- Built-in commands: help, ls, echo, clear, mem, ps, reboot, uname
- Command parsing with basic argument tokenization
- Interactive prompt with keyboard input via the VGA/keyboard driver stack

```
nimbu@localhost:~$ uname -a
NIMBU OS v1.0 | x86 | Kernel Build: 2026-01-15
nimbu@localhost:~$ mem
Total RAM : 128 MB | Used: 4.2 MB | Free: 123.8 MB
```

3. Technical Implementation

3.1 Build Environment

Host OS	Ubuntu 22.04 LTS (x86_64)
Cross Compiler	GCC i686-elf-gcc 12.x (freestanding)
Assembler	NASM 2.15 (x86 assembly)
Linker	GNU LD with custom linker script (kernel.ld)
Build System	GNU Make
Emulator / Testing	QEMU 7.x (qemu-system-i386)
Debugger	GDB with QEMU remote stub
Version Control	Git

3.2 Directory Structure

```
nimbu-os/
├── boot/              # Bootloader (boot.asm, boot_menu.asm)
├── kernel/
│   ├── arch/x86/      # CPU-specific: GDT, IDT, interrupts
│   ├── mm/            # Memory management (pmm.c, vmm.c, heap.c)
│   ├── proc/          # Process management (scheduler.c, pcb.c)
│   ├── drivers/       # Hardware drivers (vga.c, kbd.c, serial.c)
│   ├── shell/         # Nimbu shell (shell.c, commands.c)
│   └── kernel.c       # Kernel entry point (kmain)
├── include/           # Kernel headers
├── scripts/           # Build scripts
├── linker.ld          # Linker script
├── Makefile           # Build system
└── README.md          # Project readme
```

3.3 Kernel Build Process

- Step 1: Assemble bootloader — nasm -f bin boot/boot.asm -o boot.bin
- Step 2: Compile kernel C sources with i686-elf-gcc -ffreestanding -nostdlib
- Step 3: Link object files using linker.ld — kernel loaded at physical 0x100000
- Step 4: Create disk image — cat boot.bin kernel.bin > nimbu.img
- Step 5: Test in QEMU — qemu-system-i386 -drive format=raw,file=nimbu.img

3.4 Key Design Decisions

Decision	Choice Made	Rationale

Architecture	x86 (i686)	Broad documentation, hardware support, and toolchain availability
Kernel Type	Monolithic	Simpler implementation; suitable for minimal OS scope
Memory Model	Higher-half kernel	Follows Linux convention; cleanly separates user/kernel space
Boot Protocol	Custom MBR + stage2	Full control over boot menu and branding (NIMBU lemon logo)
Scheduling	Round-robin (preemptive)	Simple, fair, and easy to implement correctly
Language	C + x86 ASM	Low-level control with manageable complexity

4. Testing & Validation

4.1 Test Strategy

All components of NIMBU OS are validated through unit-level tests (where possible), integration tests in QEMU, and manual validation of outputs. Given the bare-metal nature of kernel development, QEMU with GDB remote debugging serves as the primary testing platform.

4.2 Test Cases

Test ID	Component	Test Description	Expected Result	Status
TC-01	Bootloader	System boots and displays NIMBU OS boot menu with lemon logo	Boot menu rendered in VGA text mode	PASS
TC-02	Memory (PMM)	Allocate 100 pages, free 50, re-allocate 50	No corruption, bitmap consistent	PASS
TC-03	Memory (VMM)	Enable paging, map virtual to physical addresses	CR3 loaded, page faults handled	PASS
TC-04	Heap	kmalloc / kfree 1000 cycles with varying sizes	No memory leaks detected	PASS
TC-05	IDT/ISR	Trigger divide-by-zero exception (int 0)	Handler prints exception, halts gracefully	PASS
TC-06	Keyboard	Press keys and verify echo on shell	Correct ASCII output on screen	PASS
TC-07	Scheduler	Create 3 processes; verify round-robin execution	All processes execute, context switches correct	PASS
TC-08	Shell	Execute all built-in commands	Correct output for each command	PASS
TC-09	Serial	Send debug output via COM1	Output visible in QEMU serial terminal	PASS
TC-10	Page Fault	Access unmapped memory from kernel	Page fault ISR triggered; system halts safely	PASS

4.3 Performance Metrics

Kernel Boot Time	< 200 ms (QEMU, 128 MB RAM)
Kernel Binary Size	~48 KB (stripped ELF)
Memory Overhead (Kernel)	~4.2 MB at runtime
Context Switch Latency	< 5 µs (measured via TSC)
Shell Latency	< 1 ms per command dispatch

5. Challenges & Learnings

5.1 Technical Challenges

- Setting up the i686-elf cross-compiler toolchain without interfering with the host GCC
- Debugging the A20 line enable sequence for accessing memory above 1 MB in real mode
- Correctly aligning the GDT descriptor entries and handling selector offsets in segment registers
- Implementing stack-based context switching with correct register preservation using naked functions
- Handling recursive page faults during early VMM initialization before the IDT was fully set up
- Synchronizing the PIC EOI (End of Interrupt) signal correctly to avoid spurious IRQs

5.2 Key Learnings

- Gained deep understanding of x86 protected mode, segmentation, and paging mechanisms
- Understood the full boot sequence: BIOS → MBR → Bootloader → Kernel entry
- Learned how Linux-style higher-half kernels work and why they are the standard
- Developed debugging skills using GDB with QEMU's remote stub (target remote :1234)
- Understood interrupt-driven I/O and the role of the PIC in hardware interrupt routing
- Appreciated the importance of memory alignment, cache coherency, and calling conventions

6. Future Enhancements

The following enhancements are planned for future iterations of NIMBU OS:

Priority	Feature	Description
High	VFS Layer	Virtual filesystem abstraction to support multiple filesystem drivers
High	EXT2 Driver	Read-only EXT2 filesystem support for loading user programs
High	ELF Loader	Parse and load ELF32 executables for user-space process execution
Medium	User Space	Ring 3 privilege separation, syscall interface, and user stacks
Medium	Network Stack	Minimal TCP/IP stack with RTL8139 driver for QEMU networking
Low	GUI	Simple pixel framebuffer GUI using VESA BIOS Extensions (VBE)
Low	SMP Support	Multi-core support using APIC and symmetric multiprocessing

7. References

#	Resource	URL / Source
1	Linux Kernel Official	https://kernel.org/
2	OSDev Wiki	https://wiki.osdev.org/
3	Intel x86 Manual Vol. 3	Intel Software Developer's Manual — System Programming Guide
4	The Little Book About OS Development	https://littleosbook.github.io/
5	James Molloy's Kernel Tutorial	http://www.jamesmolloy.co.uk/tutorial_html/
6	NASM Documentation	https://nasm.us/doc/
7	QEMU Documentation	https://www.qemu.org/docs/master/
8	GCC Cross-Compiler Guide	https://wiki.osdev.org/GCC_Cross-Compiler
9	ZOHO SETU Project List	Software_Jan_2026_SETU_Project_List.pdf — Project #10

8. Declaration

We hereby declare that the project NIMBU OS (Project No. 10) submitted to ZOHO SETU is our original work carried out as part of the SETU project-based internship initiative by Zoho Nagpur. All references and resources used have been duly acknowledged. The implementation, documentation, and analysis presented in this report are our own.

Project Name	NIMBU OS — Baseline Kernel OS
SETU Project No.	10
Team / Student Name(s)	Someshwar Ravindra Mankar
College / Branch	GCOE Amravati , 3rd year CSE
	_____
	_____
	_____

NIMBU OS — Clean, Minimal, Zesty